Guia de Comandos

Gincana Linux: todos os comandos necessários, por categoria

Este guia reúne os comandos usados nas perguntas e nas corridas da gincana, organizados nas cinco categorias do jogo (navegação, arquivos, textos, permissões e pacotes), mais alguns comandos gerais. Os exemplos usam nomes inventados (fotos, lista.txt, frutas.txt), então adapte aos nomes que aparecerem na missão. No final há dicas para trabalhar mais rápido no terminal.

1. Navegação

Em todos os comandos, o que vem depois do nome do comando (pasta, arquivo, palavra) é o argumento.

Comando	O que faz	Exemplo
`pwd`	Mostra o caminho completo da pasta onde você está.	`pwd -> /home/maria/fotos`
`ls`	Lista o que existe na pasta atual.	`ls`
`ls pasta`	Lista outra pasta sem entrar nela.	`ls fotos`
`ls -a`	Lista também os arquivos escondidos (nomes que começam com ponto).	`ls -a`
`ls -l`	Lista com detalhes: permissões, dono, tamanho e data.	`ls -l`
`ls -la`	Combina os dois: detalhes e arquivos escondidos.	`ls -la`
`ls -R`	Lista também o conteúdo das subpastas.	`ls -R fotos`
`cd pasta`	Entra em uma pasta.	`cd fotos`
`cd ..`	Volta uma pasta.	`cd ..`
`cd ~ (ou só cd)`	Vai direto para a sua pasta pessoal (home), de qualquer lugar.	`cd ~`
`cd ~/pasta`	Vai direto para uma pasta dentro da sua home.	`cd ~/documentos`
`cd -`	Volta para a pasta em que você estava antes.	`cd -`
`find onde -name "nome"`	Procura um arquivo pelo nome, na pasta e em todas as subpastas dela.	`find ~ -name "receita.txt"`

2. Manipulação de arquivos

Atenção: o terminal não tem lixeira. O que o rm apaga não volta.

Comando	O que faz	Exemplo
`mkdir pasta`	Cria uma pasta nova.	`mkdir fotos`
`mkdir pasta1 pasta2`	Cria várias pastas de uma vez.	`mkdir docs logs`
`mkdir -p a/b/c`	Cria pastas dentro de pastas, de uma vez só.	`mkdir -p fotos/2026/ferias`

Comando	O que faz	Exemplo
`touch arquivo`	Cria um arquivo vazio.	`touch lista.txt`
`cp origem destino`	Copia um arquivo (o original continua no lugar).	`cp lista.txt fotos` `cp lista.txt copia.txt`
`cp -r pasta destino`	Copia uma pasta inteira, com tudo que há dentro.	`cp -r fotos backup`
`mv origem pasta`	Move um arquivo para dentro de uma pasta.	`mv lista.txt fotos`
`mv antigo novo`	Renomeia um arquivo (ou uma pasta).	`mv lista.txt compras.txt`
`rm arquivo`	Apaga um arquivo para sempre.	`rm lista.txt`
`rm -r pasta`	Apaga uma pasta inteira, com tudo que há dentro.	`rm -r fotos`
`rm -rf pasta`	Igual ao anterior, mas à força e sem perguntar. Use com muito cuidado.	`rm -rf fotos`
`mv *.txt docs`	Usa o coringa * para mover todos os arquivos .txt de uma vez.	`mv *.txt docs` `mv *.log logs`

3. Manipulação de textos

Comando	O que faz	Exemplo
`cat arquivo`	Mostra todo o conteúdo de um arquivo de texto.	`cat receita.txt`
`less arquivo`	Abre o arquivo página por página. Sai com a tecla q.	`less servidor.log`
`head -n N arquivo`	Mostra as N primeiras linhas.	`head -n 3 lista.txt`
`tail -n N arquivo`	Mostra as N últimas linhas.	`tail -n 3 lista.txt`
`wc -l arquivo`	Conta quantas linhas o arquivo tem.	`wc -l lista.txt`
`grep palavra arquivo`	Mostra só as linhas que contêm a palavra.	`grep banana frutas.txt`
`grep -n palavra arquivo`	Igual ao grep, mas mostra o número de cada linha encontrada.	`grep -n banana frutas.txt`
`grep -c palavra arquivo`	Em vez de mostrar as linhas, conta quantas foram encontradas.	`grep -c banana frutas.txt`
`grep -i palavra arquivo`	Ignora a diferença entre maiúsculas e minúsculas.	`grep -i banana frutas.txt`
`echo "texto"`	Mostra um texto na tela.	`echo "olá, mundo"`
`comando > arquivo`	Guarda o resultado de um comando em um arquivo. Se o arquivo já existir, o conteúdo antigo é APAGADO.	`grep banana frutas.txt > bananas.txt` `echo "oi" > nota.txt`
`comando >> arquivo`	Igual ao anterior, mas ACRESCENTA no final, sem apagar nada.	`echo "mais uma" >> nota.txt`
`comando1 | comando2`	O pipe: entrega o resultado do primeiro comando para o segundo.	`ls | wc -l` `grep banana frutas.txt | wc -l`

4. Permissões

Comando	O que faz	Exemplo
`ls -l`	Mostra as permissões de cada arquivo, por exemplo -rwxr-xr-x (veja a explicação abaixo).	`ls -l segredo.txt`
`chmod u+r arquivo`	Dá ao dono (u) o poder de LER o arquivo.	`chmod u+r segredo.txt`
`chmod +x arquivo`	Dá o poder de EXECUTAR o arquivo (necessário para scripts).	`chmod +x meu_script.sh`
`chmod -x arquivo`	Tira o poder de executar.	`chmod -x meu_script.sh`
`chmod 755 arquivo`	Define as permissões com números: dono rwx, grupo e outros r-x.	`chmod 755 meu_script.sh`
`./script`	Executa um script que está na pasta atual. O ./ quer dizer "aqui, nesta pasta".	`./meu_script.sh`
`sudo comando`	Executa o comando com poderes de administrador (pede a sua senha).	`sudo apt update`

Como ler as permissões. Em `-rwxr-xr-x`, o primeiro caractere é o tipo (- arquivo, d pasta). Depois vêm três trios de letras: **dono** (u), **grupo** (g) e **outros** (o). Em cada trio, **r** = ler, **w** = escrever, **x** = executar, e um traço significa sem aquele poder. Nos números do chmod, r vale 4, w vale 2 e x vale 1, e cada dígito soma os poderes de um trio (7 = 4+2+1 = rwx, 5 = 4+1 = r-x).

5. Gerenciamento de pacotes

Os comandos mudam conforme a família da distro. Instalar e remover pacotes exige sudo.

Ação	Debian, Ubuntu, Mint (apt)	Fedora (dnf)	Arch, Manjaro (pacman)
Atualizar a lista de pacotes (catálogo)	`sudo apt update`	`sudo dnf check-update`	`sudo pacman -Sy`
Procurar um pacote	`apt search pacote`	`dnf search pacote`	`pacman -Ss pacote`
Instalar um pacote	`sudo apt install pacote`	`sudo dnf install pacote`	`sudo pacman -S pacote`
Remover um pacote	`sudo apt remove pacote`	`sudo dnf remove pacote`	`sudo pacman -R pacote`
Atualizar os programas instalados	`sudo apt upgrade`	`sudo dnf upgrade`	`sudo pacman -Syu`

Diferença entre update e upgrade (apt). O **`update`** só atualiza a lista de pacotes disponíveis (o catálogo). O **`upgrade`** atualiza os programas que você já tem instalados. Para ver onde um programa foi instalado (e confirmar que ele existe), use **`which programa`**, por exemplo **`which cowsay`**. Dica: **`apt search pacote | grep pacote`** filtra a enxurrada de resultados da busca.

6. Comandos gerais (Rodada Relâmpago e desempate)

Comando	O que faz	Exemplo
`man comando`	Abre o manual completo de um comando. Sai com a tecla q.	`man ls`
`comando --help`	Mostra um resumo rápido de como usar o comando.	`ls --help`
`clear (ou Ctrl+L)`	Limpa a tela do terminal.	`clear`
`history`	Lista os comandos que você já digitou.	`history`
`whoami`	Mostra o nome do seu usuário.	`whoami`
`date`	Mostra a data e a hora.	`date`
`df -h`	Mostra o espaço livre e usado nos discos, em unidades legíveis.	`df -h`
`top (ou htop)`	Mostra os processos em execução em tempo real. Sai com a tecla q.	`top`

7. Dicas para o terminal

Truques que economizam tempo (e valem pontos na gincana).

Dica	Como usar
TAB completa nomes	Digite o começo do nome de um arquivo ou pasta e aperte **TAB**: o terminal completa sozinho. Se houver mais de uma opção, aperte TAB duas vezes para ver a lista. Ex.: **`cd Docu`** + TAB vira **`cd Documentos/`**. Além de rápido, evita erros de digitação.
Setas para cima e para baixo	A **seta para cima** traz o comando anterior; apertando de novo, você volta nos comandos mais antigos, e a **seta para baixo** avança. Para repetir um comando, aperte seta para cima e Enter. Também dá para editar o comando antes de executar (use as setas para esquerda e direita).
Coringa * (todos os arquivos)	O **`*`** vale por qualquer quantidade de caracteres. Serve para pegar todos os arquivos de uma extensão: **`ls *.txt`**, **`mv *.log logs`**, **`cp *.jpg fotos`**. Também funciona com o começo do nome: **`rm foto*`** apaga tudo que começa com foto. O **`?`** vale por um único caractere (**`ls foto?.jpg`**). Antes de apagar com coringa, rode o mesmo padrão com ls para ver o que será afetado.
Atalhos que ajudam	**Ctrl+C** cancela um comando que travou. **Ctrl+L** limpa a tela. **Ctrl+A** vai para o início da linha e **Ctrl+E** para o fim. **Ctrl+U** apaga a linha inteira. **Ctrl+R** busca no histórico: digite parte de um comando antigo. **`!!`** repete o último comando (ex.: **`sudo !!`** repete com sudo).
Atalhos de caminho	**`.`** é a pasta atual, **`..`** é a pasta acima, **`~`** é a sua home e **`/`** é a raiz do sistema. **`cd -`** volta para a pasta anterior.
Cuidado com o rm	Não existe lixeira. Antes de apagar, confira com **`ls`**. Para o terminal pedir confirmação, use **`rm -i`**. Nunca rode **`rm -rf`** sem ter certeza do caminho.
Maiúsculas importam	No Linux, **`Arquivo.txt`** e **`arquivo.txt`** são arquivos diferentes. Isso vale para comandos e para palavras no grep (use **`grep -i`** para ignorar a diferença).
Nomes com espaço	Coloque o nome entre aspas: **`cd "Meus Documentos"`**. Sem as aspas, o terminal entende dois nomes separados. O melhor é evitar espaços ao criar arquivos.

Dica	Como usar
Combine comandos	**`comando1 && comando2`** roda o segundo só se o primeiro der certo (ex.: **`mkdir docs && cd docs`**). O **`;`** roda um depois do outro, mesmo se o primeiro falhar.
Copiar e colar	No terminal, o copiar é **Ctrl+Shift+C** e o colar é **Ctrl+Shift+V**. O Ctrl+C sozinho cancela o comando em execução.
Leia as mensagens de erro	*No such file or directory* significa que o nome ou o caminho está errado (confira com ls). *Permission denied* significa que falta permissão (veja ls -l e chmod, ou use sudo). *command not found* significa que o comando não existe ou não está instalado.
Saindo de telas presas	Em man, less e top, saia com a tecla **q**. Se o terminal parecer travado, tente **Ctrl+C**.
Para a gincana	Use TAB e as setas o tempo todo: ganham segundos preciosos. Leia o texto da missão com calma antes de digitar. Teste com ls antes de apagar ou mover, e só levante a mão depois de conferir o resultado na tela.